OPERATIONS DOCUMENTATION

Library System — Deployment Runbook

Deploying, verifying, and operating a production instance

Applies to — Flask app, Python 3.12, Postgres **Repository** — github.com/uscabayaosj/LibrarySystem

0 What this app needs from a host

Two requirements drive every choice in this document:

1. **A real database.** SQLite is a single file. It works locally, but on a host with an ephemeral filesystem it is wiped on every deploy and not shared between instances. Use managed Postgres in production.
2. **A persistent disk.** *Admin → Settings* writes the uploaded logo and its generated icon set to `static/uploads/branding/`. Without a persistent volume, the logo disappears on the next deploy.

> **HOST COMPATIBILITY**
>
> **Container/VM hosts (Render, Railway, Fly.io) fit as-is. Serverless platforms (Vercel, Netlify, Lambda) do not** — their filesystem is read-only apart from a temp directory that doesn't survive between invocations, which breaks both requirements. Running there would need Postgres *and* logo storage moved to an object store (S3, Vercel Blob, R2) — a code change to `logo_upload.py`, not just configuration.

The two commands a host needs

Build	`pip install -r requirements.txt`
Start	`python -c "from app import init_db; init_db()" && gunicorn app:app --bind 0.0.0.0:$PORT --timeout 60`

The start command migrates the database before gunicorn binds, so the schema is current before the first request is served. The explicit `--bind 0.0.0.0:$PORT` matters: gunicorn otherwise binds to `127.0.0.1`, which the host can't route to.

Worker count is deliberately not set. Gunicorn reads `WEB_CONCURRENCY`, which hosts set from the instance's available CPU (Render does this automatically), so leaving it off means the app scales with the instance instead of overriding it with a hardcoded number that could exhaust memory on a small plan.

Hosts that read a `Procfile` (Railway, Fly, Heroku) can use the one in the repo root instead — it splits the same work into a `release` and a `web` step.

1 First deploy — Render

The repo ships `render.yaml`, which declares the web service, the Postgres database, and the persistent disk together.

Blueprint (recommended)

Dashboard → **New → Blueprint** → pick this repo → Apply. Then set `ADMIN_PASSWORD` in the service's Environment tab before the first boot (see step 2). Everything else is already declared.

Manual setup

Dashboard → **New → Web Service** → pick this repo, then:

Runtime	Python 3
Build Command	`pip install -r requirements.txt`

Start Command	`python -c "from app import init_db; init_db()" && gunicorn app:app --bind 0.0.0.0:$PORT --timeout 60`
Health Check Path	`/login`

OVERWRITE THE PRE-FILLED BUILD COMMAND

Render detects `poetry.lock` and suggests `poetry install`, which pulls in dev dependencies the production image doesn't need. It no longer *fails* — `package-mode = false` in `pyproject.toml` handles that — but `pip install -r requirements.txt` is the leaner build.

Then, before deploying:

1. **New → Postgres.** Create the database, copy its **Internal Database URL**.
2. **Environment tab** — add the variables in the table below.
3. **Disks tab** — add a disk, mount path `/opt/render/project/src/static/uploads`, 1 GB.

Variable	Value
`DATABASE_URL`	The Postgres URL from step 1. The `postgres://` form is fine; the app rewrites it to `postgresql://` at startup.
`SECRET_KEY`	A long random string. Generate with `python -c "import secrets; print(secrets.token_urlsafe(48))"`. **The app refuses to start in production without it.**
`FLASK_ENV`	`production`
`ADMIN_PASSWORD`	The password for the seeded admin account.
`PYTHON_VERSION`	`3.12`

MOST COMMON MISTAKE

Skipping the persistent disk. Everything appears to work until the first redeploy, when the uploaded logo vanishes.

2 First boot

The first start runs every migration and seeds one admin account:

- Username `admin`, password `$ADMIN_PASSWORD` (or `admin` if unset).

Sign in and change that password immediately. The seed only happens when no admin exists, so it will not silently re-create or reset the account later.

3 Verify the deploy

```
curl -sf https://YOUR-APP.onrender.com/login  > /dev/null && echo "login OK"
curl -sf https://YOUR-APP.onrender.com/manifest.json | head -c 200
```

Check which database it actually connected to

The migration log line names the backend:

- `Context impl PostgresqlImpl` — correct.
- `Context impl SQLiteImpl` — `DATABASE_URL` didn't take effect. The app will run perfectly and lose everything on the next deploy. Fix this before entering real data. Startup also prints an unmissable

warning banner in this case.

Then in a browser

- [ ] Sign in as admin
- [ ] **Admin → Settings** — set the organization name, upload a logo, pick a theme color, save
- [ ] Hard-refresh: the sidebar shows the logo and name, and the accent color changed
- [ ] Add a book, register a member account, borrow it
- [ ] On a phone: the bottom tab bar appears, and **Add to Home Screen** shows the uploaded logo as the app icon

4 Subsequent deploys

Push to `main`. The host rebuilds and re-runs the start command, which applies any new migrations before serving. Nothing manual.

After changing a model, generate the migration locally and commit it with the code:

```
FLASK_APP=app.py flask db migrate -m "describe the change"
FLASK_APP=app.py flask db upgrade      # apply locally
```

> **TREAT THE MIGRATION AS PART OF THE CHANGE**
> A deploy whose migration file is missing will start fine and then fail at runtime on the missing column — not at build, and not at boot.

5 Rollback

Render → **Deploys** tab → pick the last good deploy → **Redeploy**.

Note that this rolls back *code*, not the *database*. Migrations don't auto-revert: if the bad deploy added a column, the rollback leaves it in place, which is harmless. If it *dropped* or rewrote one, restore the database from a backup instead (Render Postgres keeps daily backups on paid plans) and consider `flask db downgrade` locally against a copy first to confirm the down-migration is correct.

6 Operations

Open a shell (Render → Shell tab)

```
# Inspect migration state
FLASK_APP=app.py flask db current
FLASK_APP=app.py flask db history
```

Reset a locked-out admin's password

```
python -c "
from app import app
from extensions import db
from models import User
with app.app_context():
    u = User.query.filter_by(username='admin').first()
    u.set_password('a-new-strong-password')
    db.session.commit()
    print('password updated')
"
```

Back up the database

Run locally, with the *External* database URL:

```
pg_dump "$EXTERNAL_DATABASE_URL" > backup-$(date +%F).sql
```

The persistent disk holds only logo files, which can be re-uploaded, so the database is the thing that actually needs backing up.

FREE-TIER CAVEAT

Render's free web services spin down after inactivity, so the first request after an idle period takes ~30 seconds. Free Postgres instances also expire after 90 days. Fine for evaluation; move to a paid instance for real use.

7 Troubleshooting

Symptom	Cause	Fix
`RuntimeError: SECRET_KEY environment variable must be set`	`FLASK_ENV=production` with no `SECRET_KEY`	Set `SECRET_KEY` in the environment
Deploy succeeds, host reports "no open ports detected"	gunicorn bound to `127.0.0.1`	Use the full start command, including `--bind 0.0.0.0:$PORT`
`Error: can't chdir to './app_dir'`, or any start command you don't recognise	The Start Command field was left empty, so the host invented one	Paste the Start Command from section 1 into the service's settings
`Can't load plugin: sqlalchemy.dialects:postgres`	Very old `DATABASE_URL` handling	Already handled — the app rewrites `postgres://`. Confirm you're on current `main`
`The current project could not be installed: No file/folder found for package library-system`	The host autodetected `poetry.lock` and ran a bare `poetry install`, which tries to install the app as a package	Already handled — `package-mode = false`. Setting the Build Command to `pip install -r requirements.txt` also avoids it
Build uses an unexpected Python version	No version pinned, so the host picks its default (Render currently defaults to 3.14)	Already handled — `.python-version` pins 3.12. `PYTHON_VERSION` overrides it
`ModuleNotFoundError: No module named 'psycopg2'`	Build didn't install requirements	Check the Build Command is `pip install -r requirements.txt`
Uploaded logo disappears after a deploy	No persistent disk	Mount a disk at `/opt/render/project/src/static/uploads`
`no such table` / `column ... does not exist` at runtime	A migration wasn't committed, or the start command doesn't migrate	Confirm the start command includes the `init_db()` prefix; generate and commit the missing migration
Sign-in appears to succeed but bounces back to `/login`	Cookie marked secure while served over plain HTTP	Serve over HTTPS (Render does this by default), or unset `SESSION_COOKIE_SECURE` for a non-TLS test host

Symptom	Cause	Fix
Startup logs `WARNING: running in production on SQLite`, or migrations log `Context impl SQLiteImpl`	`DATABASE_URL` isn't set, so the app is writing to a local file that the next deploy destroys	Attach managed Postgres and set `DATABASE_URL`. Any data created meanwhile is lost on the next deploy
Data disappeared after a deploy	Same cause as above — the app was on SQLite, not Postgres	Set `DATABASE_URL` before putting real data in

8 Reference

Environment variables

`SECRET_KEY`	Required in production
`DATABASE_URL`	Default `sqlite:///library.db`
`FLASK_ENV`	Default `development`
`FLASK_DEBUG`	Default `0` — never `1` in production
`SESSION_COOKIE_SECURE`	`1` in production
`ADMIN_PASSWORD`	Seeds first admin only

Migration states handled on boot

Brand-new DB	Runs every migration
Already migrated	Applies only what's outstanding
Legacy `create_all()`	Stamped at baseline, then upgraded — existing rows preserved

Migrations use `render_as_batch`, so column changes work on SQLite and Postgres alike.

Running tests

```
pip install -r requirements.txt pytest
pytest
```

CI (`.github/workflows/tests.yml`) runs the same suite on every push and pull request against `main`, on Python 3.11 and 3.12.

Library Management System — deployment runbook. Generated from the project README; the repository remains the source of truth.